Measuring Success in the AEP Agentic Platform

Decode.ai • April 2026

Adobe

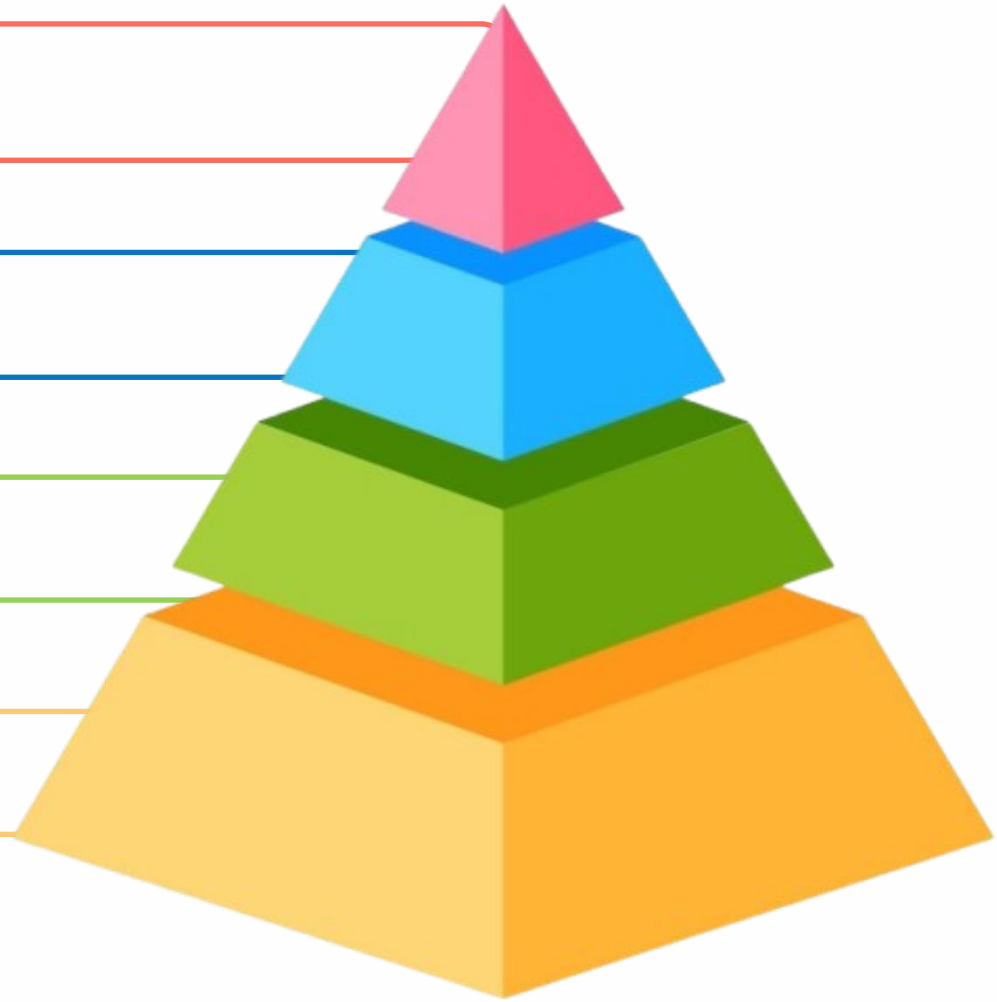
The initial quality model

END 2 END TESTS

INTEGRATION TESTS

FUNCTIONAL TESTS

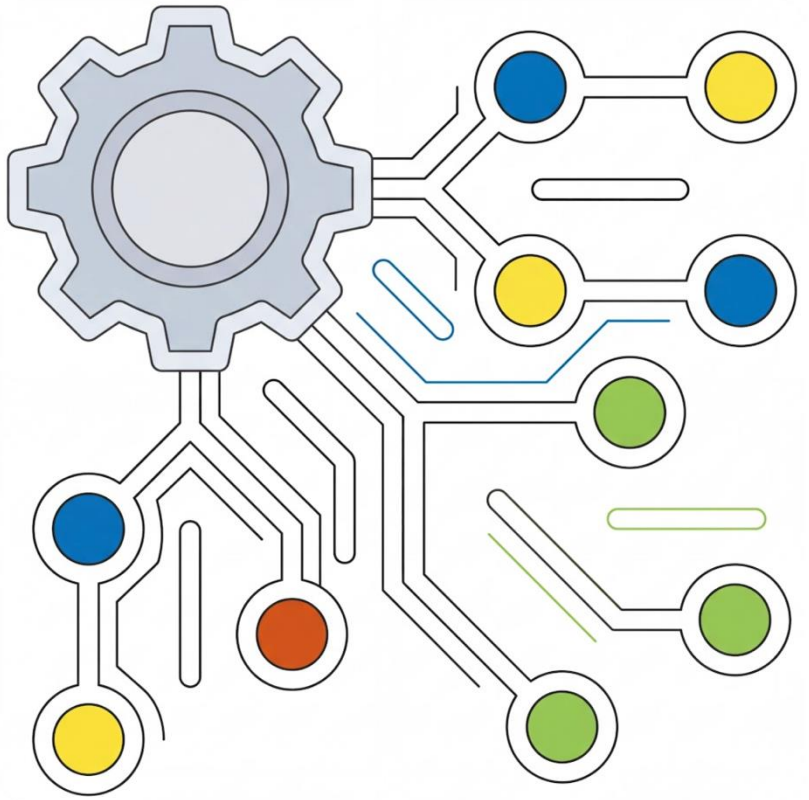
UNIT TESTS



Software testing always relied on a familiar structure: unit tests, integration and functional tests, and end-to-end tests.

The new reality is non-determinism

- Agents reason
- They choose tools
- They may reorder steps based on context
- They respond differently to the same input
- They operate over multiple turns
- Their behavior changes not only when we update code, but also when we adjust a prompt or when the model provider rolls out a new version.



The traditional testing approach no longer supports the quality quest

A single "correct answer" often doesn't exist, and failures can be subtle, plausible, and difficult to detect.

Adobe’s Agentic Experience Platform in numbers

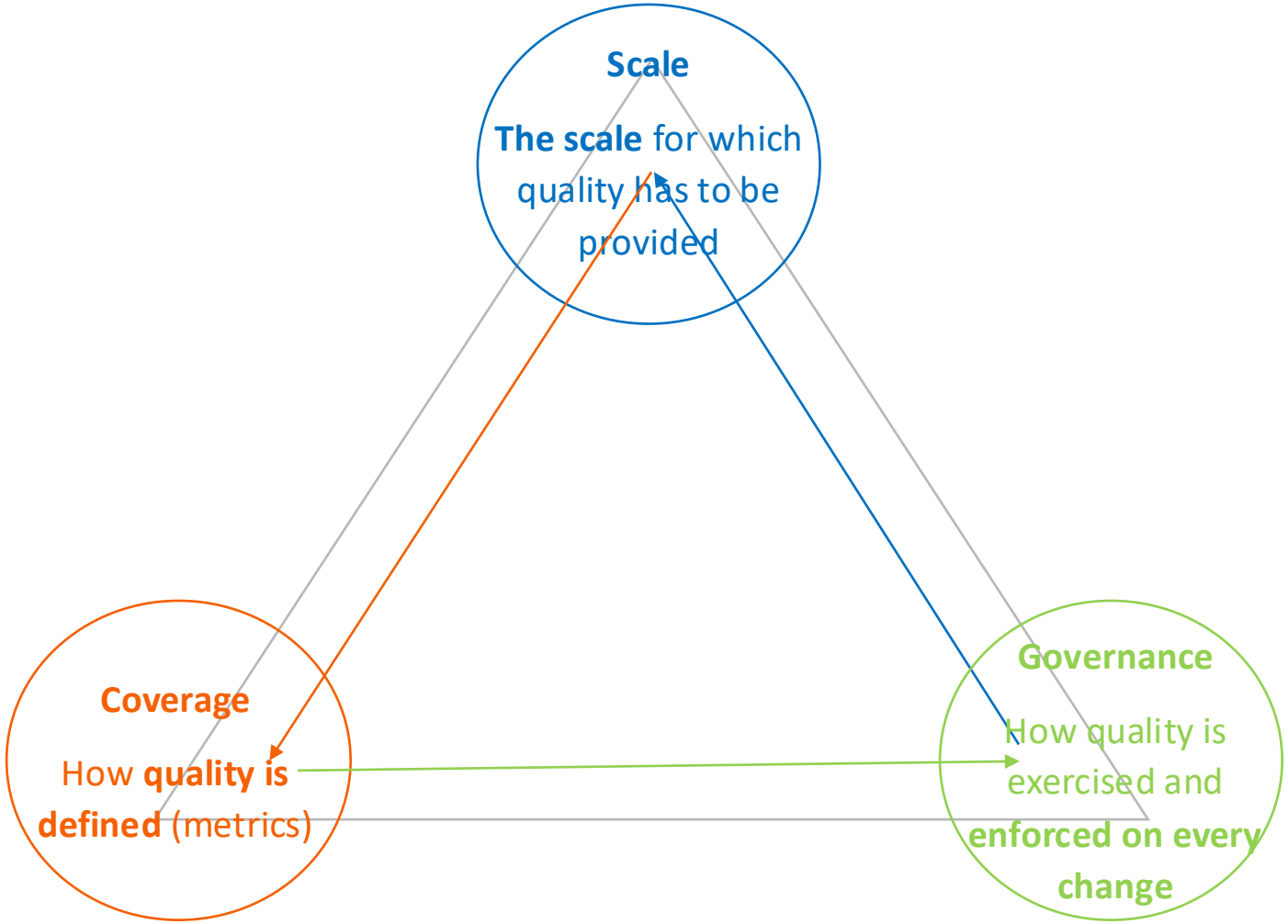
- **100+ existing agents**
  - Content & Creative
  - Journeys & Engagement
  - Audience & Personalization
  - Data & Intelligence
  - Platform & Operations

- **Velocity**
  - +1–5 new agents/week

- **Provider**
  - OpenAI
  - Anthropic

- **Distributed Tracing Volume**
  - ~ 0.5 TB per day

Where do agentic evaluations have challenges?



The goal here is to make quality operable at the throughput of hundreds of agents

Quality @ scale – How do we test terabytes of conversations?

Volume
N Agents x M conversations

- The agents are instrumented via Otel
- Collect only what's needed

Standardization
Same format, same rules

- The data is persisted in AEP Datalake based on a standard shared schema (XDM)
- Standardized multi team access

Cost
Storage + Compute

- Persist only once, reuse as needed
- Data is sampled

Shared persistence layer: AEP Datalake

The metrics we use – the common taxonomy for failure

Risk	Failure Type	Example
Trust	Hallucination	*Agent invents a policy that doesn't exist*
Trust	Factual error	*Wrong price, wrong date, wrong status*
Safety	PII leak	*Agent echoes back credit card number*
Safety	Harmful content	*Toxic, biased, or dangerous response*
Safety	False positive	*Blocks "kill the order" as violent*
Routing	Wrong agent	*The input is sent to the wrong agent*
UX	Intent misread	*"Cancel order" → agent tries to place new order*
UX	Loop / no resolution	*10 turns, user gives up*
UX	Poor tone	*Too formal, too casual, robotic*
Reliability	Tool failure	*API call fails, agent doesn't recover*
Reliability	Timeout	*Agent hangs, no response*
Coverage	Unknown intent	*Agent doesn't know what to do, fails silently*

We define the evaluation metrics starting from the product risks (what hurts the business) with the outcome in mind

How quality is defined

Define
What matters

- We define the evaluation metrics starting from the product risks (what hurts users and the business)
- The scoring guide and the thresholds are defined upfront

Score
Layered by cost

- Programmatic scorers (fast)
- LLM as judges
- Humans (gold datasets + calibration)

Govern
How to trust

- Validate judges against human labels before you trust them at scale
- Express quality gates clearly
- Everything is versioned -> reproducibility
- Audit trails

One scoring guide for all scorers
“Pass” means the same thing for LLMs, humans and code

How quality is exercised - Terminology

- **Test case**: Input + expected behavior
- **Pass rate (ρ)**: % of tests where expectations met
- **Behavioral diff (Δρ)**: ρ(new) − ρ(baseline)
- **Property:** Quality dimension (i.e. safety, accuracy, etc)
- **Expectation:** Threshold for a property (i.e. safety ≥ 1.0)

How quality is exercised - Why code version control isn't enough

Code versioning (Git)	Behavior versioning (Governed Behavioral Baseline)
Tracks: what changed in code	Tracks: what changed in behavior
Assumes: same input → same output	Handles: non-deterministic outputs
Diff: lines added/removed	Diff: Δρ (behavioral regression)

Why agents break traditional version control

Code change ≠behavior change

Quality is multidimensional

Behavior emerges from prompts + models + tools

How quality is exercised and enforced

Baseline
The contract

- GBB (Governed Behavioral Baseline): versioned tests + expectations (Enabling behavioral version control)
- Version control all eval logic (Git)
- Promote production failures into the baseline

Compare
Ship decisions bind to evidence

- Run baseline vs. candidate (same GBB)
- Report Δρ + per-property(not just vibe score)
- Targeted regression when full suite too costly
- Run GBB on a schedule against prod to detect drift

Enforce
Act on results

- Gates: do not merge if regression or safety fail
- Post results to PR (visible to reviewers)
- Log what ran (audit + reproducibility)
- Track experiments centrally

Reproducibility: same test + same version = same result

Adapting to the new paradigm

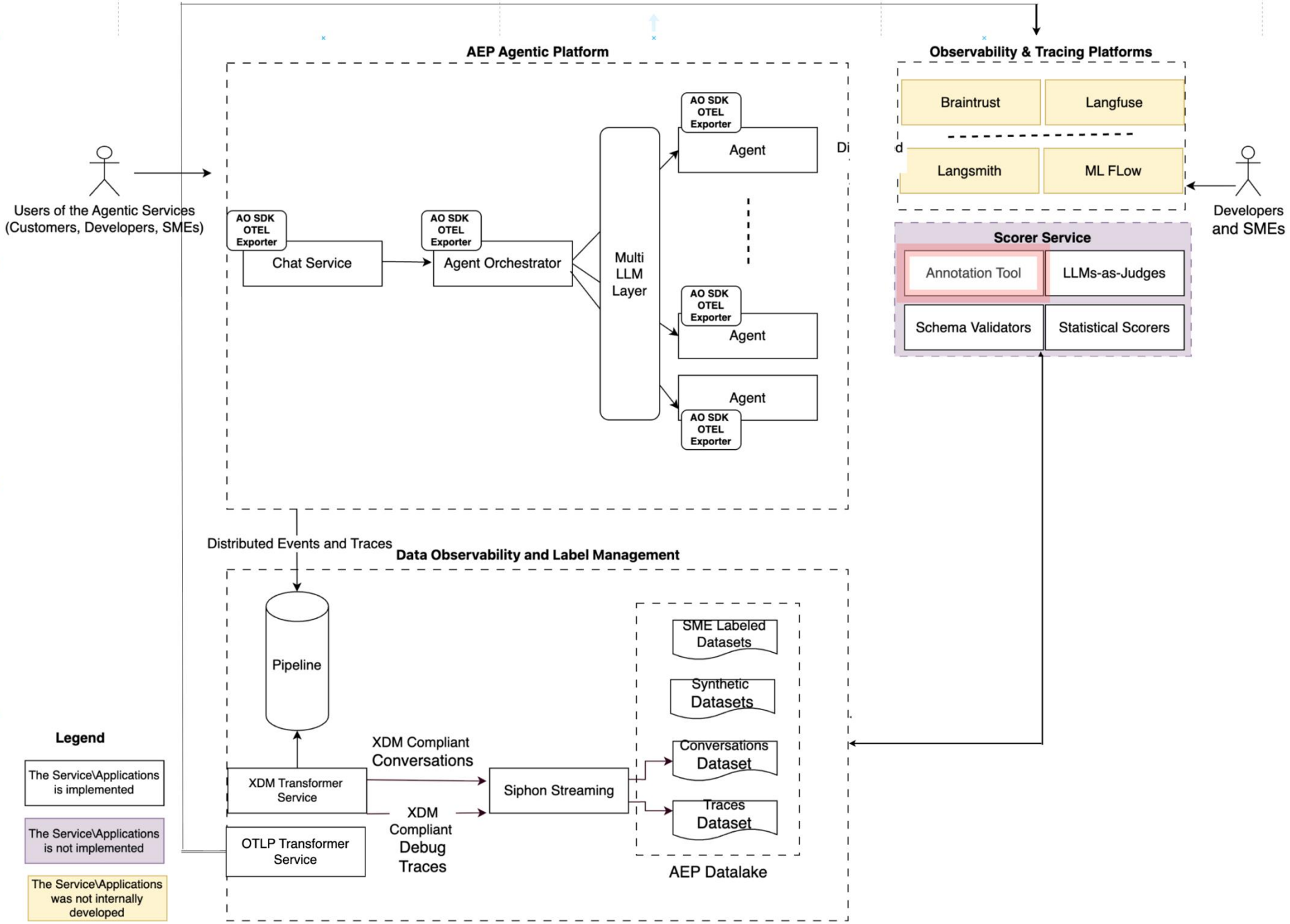
This is our approach - We ship the behavior we measured

- Behavioral version control
- A versioned baseline
- Property-based scoring
- Gates that block regressions


Adobe

What audit enables

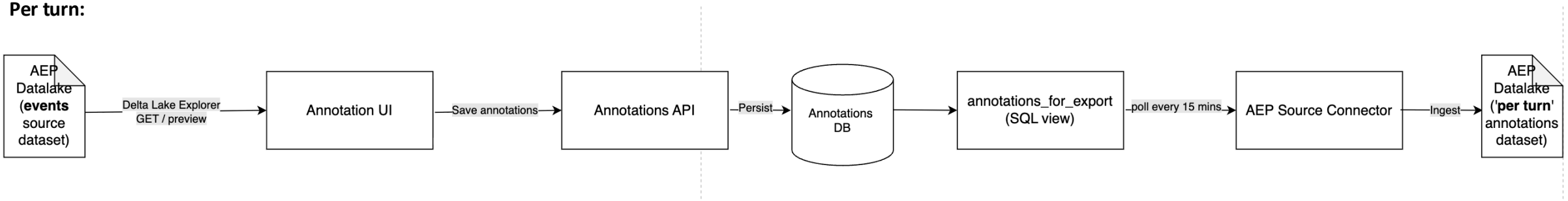
Question	How the audit answers it
"Why did we ship this version?"	*Gate passed, here's the evidence …*
"When did helpfulness start dropping?"	*Compare runs over time*
"What scorer was used for that result?"	*Logged config with version*
"Who approved this GBB change?"	*Change log with user*
"Can we recreate the Jan 15 eval?"	*Yes — all inputs are logged*
"What failed in that blocked PR?"	*Per-test outcomes*



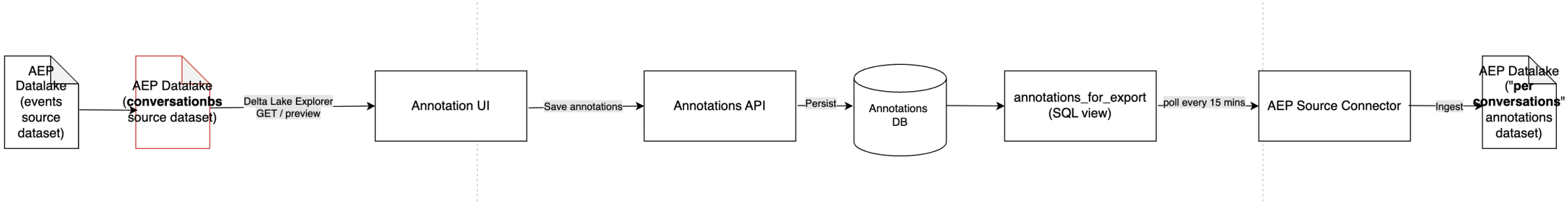
© 2024 Adobe. All Rights Reserved. Adobe Confidential.

The Annotation Flow

Per turn:



Per conversation:



© 2024 Adobe. All Rights Reserved. Adobe Confidential.